1. Environment Variables

```
ITEC_BASE_URL_V1=https://pay.itecpay.rw/api
ITEC_CALLBACK_SECRET=any_long_random_string

# Per-provider API keys
ITEC_MOMO_API_KEY=your_mtn_momo_api_key
ITEC_AIRTEL_API_KEY=your_airtel_api_key

# Reconciliation (V1 has no verify endpoint)
RECONCILE_INITIAL_DELAY_MS=20000
RECONCILE_INTERVAL_MS=15000
RECONCILE_MAX_ATTEMPTS=16
```

2. Payload Structures

MoMo payment

```
{
  "amount": 1000,
  "phone": "0788123456",
  "key": "YOUR_API_KEY",
  "req_ref": "ORDER-12345",
  "note": "Payment for Order #12345",
  "message": "Thank you for your purchase"
}
```

- V2 adds POST /api2/verify

- MoMo callback

```
{
  "status": "SUCCESS",
  "data": {
    "transaction_id": "TXN-112233",
    "req_ref": "ORDER-12345",
    "amount": 1000,
    "status": "completed"
  }
}
```

example usage codes in nodejs

src/config.js

```
require('dotenv').config();

module.exports = {
  port: parseInt(process.env.PORT || '3000', 10),
  itec: {
    baseUrl: process.env.ITEC_BASE_URL_V1,
    callbackSecret: process.env.ITEC_CALLBACK_SECRET,
    keys: {
      MTN_MOMO: process.env.ITEC_MOMO_API_KEY,
      AIRTEL:   process.env.ITEC_AIRTEL_API_KEY,
    },
  },
  reconcile: {
    initialDelayMs: +process.env.RECONCILE_INITIAL_DELAY_MS || 20000,
    intervalMs:    +process.env.RECONCILE_INTERVAL_MS    || 15000,
    maxAttempts:   +process.env.RECONCILE_MAX_ATTEMPTS   || 16,
  },
};

/** Helper — pick the right key for a provider */
module.exports.keyFor = function keyFor(provider) {
  const key = module.exports.itec.keys[provider];
  if (!key) throw new Error(`No API key configured for provider: ${provider}`);
  return key;
};
```

src/phoneUtils.js

```
/**
 * Rwandan mobile prefixes:
 *   MTN MoMo : 078, 079
 *   Airtel   : 072, 073
 */
const MTN_PREFIXES    = ['078', '079'];
const AIRTEL_PREFIXES = ['072', '073'];

function normalize(phone) {
  if (!phone) return null;
  let p = String(phone).replace(/\D/g, '');       // digits only
  if (p.startsWith('250')) p = '0' + p.slice(3);  // 250788… → 0788…
  if (p.length === 9 && p.startsWith('7')) p = '0' + p;
  return p.length === 10 && p.startsWith('07') ? p : null;
}

function detectProvider(phone) {
  const p = normalize(phone);
  if (!p) return null;
  const prefix = p.slice(0, 3);
  if (MTN_PREFIXES.includes(prefix))    return 'MTN_MOMO';
  if (AIRTEL_PREFIXES.includes(prefix)) return 'AIRTEL';
  return null;
}

function isSupported(phone) {
  return ['MTN_MOMO', 'AIRTEL'].includes(detectProvider(phone));
}
```

```
module.exports = { normalize, detectProvider, isSupported };
```

src/itecClient.js — low-level API

```
const axios = require('axios');
const cfg = require('./config');

const http = axios.create({
  baseURL: cfg.itec.baseUrl,
  timeout: 20000,
  headers: { 'Content-Type': 'application/json' },
});

/** POST /api/pay — V1 mobile money charge */
async function payMobile({ amount, phone, provider }) {
  const { data } = await http.post('/pay', {
    amount,
    phone,
    key: cfg.keyFor(provider),   // ← provider-specific key
  });
  return data;
}

/** POST /api/transfer — V1 cashout */
async function cashout({ amount, phone, provider }) {
  const { data } = await http.post('/transfer', {
    amount,
    phone,
    key: cfg.keyFor(provider),   // ← provider-specific key
```

```
  });
  return data;
}

/** POST /api/report — reports are per-key, so we must pass provider */
async function getReport({ start, end, report = 'payments', provider }) {
  const { data } = await http.post('/report', {
    key: cfg.keyFor(provider),   // ← provider-specific key
    start,
    end,
    report,
  });
  return data;
}

module.exports = { payMobile, cashout, getReport };
```

src/paymentService.js

```
const crypto = require('crypto');
const { payMobile, cashout } = require('./itecClient');
const { normalize, detectProvider, isSupported } = require('./phoneUtils');
const store = require('./store');
const reconciler = require('./reconciler');

function newRef(prefix = 'ORD') {
  return `${prefix}-${Date.now()}-${crypto.randomBytes(3).toString('hex').toUpperCase()}`;
}

async function initiatePayment({ amount, phone }) {
  const normPhone = normalize(phone);
```

```
  if (!normPhone)          throw new Error('Invalid phone. Use format 07XXXXXXXX');
  if (!isSupported(phone)) throw new Error('Only MTN MoMo (078/079) and Airtel (072/073) are supported');

  const provider = detectProvider(normPhone);   // 'MTN_MOMO' | 'AIRTEL'
  const localRef = newRef();

  store.create(localRef, { type: 'payment', provider, amount, phone: normPhone });

  const itecResp = await payMobile({ amount, phone: normPhone, provider });
  store.update(localRef, { itecInitResponse: itecResp });

  reconciler.startReconciliation(
    localRef,
    { phone: normPhone, amount, provider, reportType: 'payments' },
    (r) => console.log(`[reconcile done] ${localRef} → ${r.status}`)
  );

  return { localRef, provider, phone: normPhone, itec: itecResp };
}

async function initiateCashout({ amount, phone }) {
  const normPhone = normalize(phone);
  if (!normPhone)          throw new Error('Invalid phone. Use format 07XXXXXXXX');
  if (!isSupported(phone)) throw new Error('Only MTN MoMo and Airtel are supported for cashout');

  const provider = detectProvider(normPhone);
  const localRef = newRef('PAY');

  store.create(localRef, { type: 'cashout', provider, amount, phone: normPhone });

  const itecResp = await cashout({ amount, phone: normPhone, provider });
  store.update(localRef, { itecInitResponse: itecResp });
```

```
  reconciler.startReconciliation(
    localRef,
    { phone: normPhone, amount, provider, reportType: 'transfers' },
    (r) => console.log(`[reconcile done] ${localRef} → ${r.status}`)
  );

  return { localRef, provider, phone: normPhone, itec: itecResp };
}

async function handleCallback(payload) {
  const phone   = String(payload.phone || payload.msisdn || '').replace(/\D/g, '');
  const amount  = Number(payload.amount);
  const rawStat = String(payload.status || payload.state || '').toUpperCase();
  const transID = payload.transID || payload.transaction_id;

  const status = ['SUCCESS', 'COMPLETED', 'PAID', 'OK'].includes(rawStat) ? 'SUCCESS'
               : ['FAILED', 'DECLINED', 'CANCELLED'].includes(rawStat)    ? 'FAILED'
               : 'PENDING';

  const hit = store.findPending({ phone, amount });
  if (hit) {
    store.update(hit.ref, { status, callbackPayload: payload, transID, source: 'callback' });
    reconciler.stop(hit.ref);
  }

  return { matched: !!hit, status, transID };
}

module.exports = { initiatePayment, initiateCashout, handleCallback };
```

src/reconciler.js

```
const cfg = require('./config');
const store = require('./store');
const { getReport } = require('./itecClient');

const active = new Map();

/** Format a Date as YYYYMMDDHHmmss for the Reports API */
function fmt(d) {
  const p = (n) => String(n).padStart(2, '0');
  return `${d.getFullYear()}${p(d.getMonth() + 1)}${p(d.getDate())}${p(d.getHours())}${p(d.getMinutes())}${p(d.getSeconds())}`;
}

/** Look for a matching transaction in the report */
function matchInReport(report, { phone, amount, afterTs }) {
  const rows = report?.data || report?.transactions || report || [];
  if (!Array.isArray(rows)) return null;

  return rows.find((row) => {
    const rowPhone  = String(row.phone || row.msisdn || '').replace(/\D/g, '');
    const rowAmount = Number(row.amount);
    const rowTime   = new Date(row.date || row.created_at || row.time || 0).getTime();
    const same = rowPhone.endsWith(phone.slice(-9)) &&
                 rowAmount === Number(amount) &&
                 (!afterTs || rowTime >= afterTs - 60000);
    return same;
  });
}
```

```
function normalizeStatus(row) {
  const raw = String(row.status || row.state || '').toUpperCase();
  if (['SUCCESS', 'COMPLETED', 'PAID', 'OK'].includes(raw)) return 'SUCCESS';
  if (['FAILED', 'CANCELLED', 'DECLINED', 'EXPIRED', 'ERROR'].includes(raw)) return
'FAILED';
  return 'PENDING';
}

function startReconciliation(localRef, { phone, amount, reportType = 'payments' }, onResolve) {
  if (active.has(localRef)) return;
  let attempt = 0;
  const startedAt = Date.now();

  const tick = async () => {
    attempt++;
    if (store.isFinal(localRef)) return stop(localRef);

    const start = fmt(new Date(startedAt - 5 * 60 * 1000));     // 5 min before
    const end   = fmt(new Date());

    try {
      const report = await getReport({ start, end, report: reportType });
      const row = matchInReport(report, { phone, amount, afterTs: startedAt });

      if (row) {
        const status = normalizeStatus(row);
        console.log(`[reconcile] ${localRef} attempt ${attempt} → ${status}`);
        if (status !== 'PENDING') {
          store.update(localRef, { status, reportRow: row, source: 'reconcile' });
          stop(localRef);
          onResolve?.({ localRef, status, row });
          return;
```

```
        }
      } else {
        console.log(`[reconcile] ${localRef} attempt ${attempt} → not yet in report`);
      }
    } catch (err) {
      console.error(`[reconcile] ${localRef} attempt ${attempt} error: ${err.message}`);
    }

    if (attempt >= cfg.reconcile.maxAttempts) {
      store.update(localRef, { status: 'EXPIRED', source: 'reconcile-timeout' });
      stop(localRef);
      onResolve?.({ localRef, status: 'EXPIRED' });
      return;
    }

    active.set(localRef, setTimeout(tick, cfg.reconcile.intervalMs));
  };

  active.set(localRef, setTimeout(tick, cfg.reconcile.initialDelayMs));
}

function stop(localRef) {
  const t = active.get(localRef);
  if (t) clearTimeout(t);
  active.delete(localRef);
}

module.exports = { startReconciliation, stop };
```

Only the getReport call changes — pass provider through:

```
// inside startReconciliation's tick():
const report = await getReport({ start, end, report: reportType, provider: opts.provider });
```

Full signature update:

```
function startReconciliation(localRef, { phone, amount, provider, reportType = 'payments' }, onResolve) {
  // ... same as before, but include `provider` when calling getReport
}
```

src/store.js

```
findPending({ phone, amount }) {
  for (const [ref, t] of txns) {
    if (t.status === 'PENDING' &&
        Number(t.amount) === Number(amount) &&
        t.phone.endsWith(String(phone).slice(-9))) return { ref, txn: t };
  }
  return null;
},
```